

C H A P T E R

19

SPECIALIZED TOOLS AND
TECHNIQUES

CONTENTS

Section 19.1 Controlling Memory Allocation	820
Section 19.2 Run-Time Type Identification	825
Section 19.3 Enumerations	832
Section 19.4 Pointer to Class Member	835
Section 19.5 Nested Classes	843
Section 19.6 union: A Space-Saving Class	847
Section 19.7 Local Classes	852
Section 19.8 Inherently Nonportable Features	854
Chapter Summary	862
Defined Terms	862

The first three parts of this book discussed aspects of C++ that most C++ programmers are likely to use at some point. In addition, C++ defines some features that are more specialized. Many programmers will never (or only rarely) need to use the features presented in this chapter.

C++ *is intended* for use in a wide variety of applications. As a result, it contains features that are particular to some applications and that need never be used by others. In this chapter we look at some of the less-commonly used features in the language.

19.1 Controlling Memory Allocation

Some applications have specialized memory allocation needs that cannot be met by the standard memory management facilities. Such applications need to take over the details of how memory is allocated, for example, by arranging for new to put objects into particular kinds of memory. To do so, they can overload the new and delete operators to control memory allocation.

19.1.1 Overloading new and delete

Although we say that we can “overload new and delete,” overloading these operators is quite different from the way we overload other operators. In order to understand how we overload these operators, we first need to know a bit more about how new and delete expressions work.

When we use a new expression:

```
// new expressions
string *sp = new string("a value"); // allocate and initialize a string
string *arr = new string[10]; // allocate ten default initialized strings
```

three steps actually happen. First, the expression calls a library function named **operator new** (or **operator new[]**). This function allocates raw, untyped memory large enough to hold an object (or an array of objects) of the specified type. Next, the compiler runs the appropriate constructor to construct the object(s) from the specified initializers. Finally, a pointer to the newly allocated and constructed object is returned.

When we use a delete expression to delete a dynamically allocated object:

```
delete sp; // destroy *sp and free the memory to which sp points
delete [] arr; // destroy the elements in the array and free the memory
```

two steps happen. First, the appropriate destructor is run on the object to which sp points or on the elements in the array to which arr points. Next, the compiler frees the memory by calling a library function named **operator delete** or **operator delete[]**, respectively.

Applications that want to take control of memory allocation define their own versions of the operator new and operator delete functions. Even though the library contains definitions for these functions, we can define our own versions of them and the compiler won’t complain about duplicate definitions. Instead, the compiler will use our version in place of the one defined by the library.



When we define the global operator `new` and operator `delete` functions, we take over responsibility for all dynamic memory allocation. These functions *must* be correct: They form a vital part of all processing in the program.

Applications can define operator `new` and operator `delete` functions in the global scope and/or as member functions. When the compiler sees a `new` or `delete` expression, it looks for the corresponding operator function to call. If the object being allocated (deallocated) has class type, the compiler first looks in the scope of the class, including any base classes. If the class has a member operator `new` or operator `delete`, that function is used by the `new` or `delete` expression. Otherwise, the compiler looks for a matching function in the global scope. If the compiler finds a user-defined version, it uses that function to execute the `new` or `delete` expression. Otherwise, the standard library version is used.

We can use the scope operator to force a `new` or `delete` expression to bypass a class-specific function and use the one from the global scope. For example, `::new` will look only in the global scope for a matching operator `new` function. Similarly for `::delete`.

The operator `new` and operator `delete` Interface

The library defines eight overloaded versions of operator `new` and operator `delete` functions. The first four support the versions of `new` that can throw a `bad_alloc` exception. The next four support nonthrowing versions of `new`:

```
// these versions might throw an exception
void *operator new(size_t);           // allocate an object
void *operator new[](size_t);         // allocate an array
void *operator delete(void*) noexcept; // free an object
void *operator delete[](void*) noexcept; // free an array

// versions that promise not to throw; see § 12.1.2 (p. 460)
void *operator new(size_t, nothrow_t&) noexcept;
void *operator new[](size_t, nothrow_t&) noexcept;
void *operator delete(void*, nothrow_t&) noexcept;
void *operator delete[](void*, nothrow_t&) noexcept;
```

The type `nothrow_t` is a struct defined in the `new` header. This type has no members. The `new` header also defines a `const` object named `nothrow`, which users can pass to signal they want the nonthrowing version of `new` (§ 12.1.2, p. 460). Like destructors, an operator `delete` must not throw an exception (§ 18.1.1, p. 774). When we overload these operators, we must specify that they will not throw, which we do through the `noexcept` exception specifier (§ 18.1.4, p. 779).

An application can define its own version of any of these functions. If it does so, it must define these functions in the global scope or as members of a class. When defined as members of a class, these operator functions are implicitly static (§ 7.6, p. 302). There is no need to declare them `static` explicitly, although it is legal to do so. The member `new` and `delete` functions must be static because they are used either before the object is constructed (operator `new`) or after it has been

destroyed (operator delete). There are, therefore, no member data for these functions to manipulate.

An `operator new` or `operator new[]` function must have a return type of `void*` and its first parameter must have type `size_t`. That parameter may not have a default argument. The `operator new` function is used when we allocate an object; `operator new[]` is called when we allocate an array. When the compiler calls `operator new`, it initializes the `size_t` parameter with the number of bytes required to hold an object of the specified type; when it calls `operator new[]`, it passes the number of bytes required to store an array of the given number of elements.

When we define our own `operator new` function, we can define additional parameters. A new expression that uses such functions must use the placement form of `new` (§ 12.1.2, p. 460) to pass arguments to these additional parameters. Although generally we may define our version of `operator new` to have whatever parameters are needed, we may not define a function with the following form:

```
void *operator new(size_t, void*); // this version may not be redefined
```

This specific form is reserved for use by the library and may not be redefined.

An `operator delete` or `operator delete[]` function must have a `void` return type and a first parameter of type `void*`. Executing a `delete` expression calls the appropriate `operator` function and initializes its `void*` parameter with a pointer to the memory to free.

When `operator delete` or `operator delete[]` is defined as a class member, the function may have a second parameter of type `size_t`. If present, the additional parameter is initialized with the size in bytes of the object addressed by the first parameter. The `size_t` parameter is used when we delete objects that are part of an inheritance hierarchy. If the base class has a virtual destructor (§ 15.7.1, p. 622), then the size passed to `operator delete` will vary depending on the dynamic type of the object to which the deleted pointer points. Moreover, the version of the `operator delete` function that is run will be the one from the dynamic type of the object.

TERMINOLOGY: NEW EXPRESSION VERSUS OPERATOR NEW FUNCTION

The library functions `operator new` and `operator delete` are misleadingly named. Unlike other `operator` functions, such as `operator=`, these functions do not overload the `new` or `delete` expressions. In fact, we cannot redefine the behavior of the `new` and `delete` expressions.

A new expression always executes by calling an `operator new` function to obtain memory and then constructing an object in that memory. A `delete` expression always executes by destroying an object and then calling an `operator delete` function to free the memory used by the object.

By providing our own definitions of the `operator new` and `operator delete` functions, we can change how memory is allocated. However, we cannot change this basic meaning of the `new` and `delete` operators.

The `malloc` and `free` Functions

If you define your own global `operator new` and `operator delete`, those functions must allocate and deallocate memory somehow. Even if you define these functions in order to use a specialized memory allocator, it can still be useful for testing purposes to be able to allocate memory similarly to how the implementation normally does so.

To this end, we can use functions named `malloc` and `free` that C++ inherits from C. These functions, are defined in `cstdlib`.

The `malloc` function takes a `size_t` that says how many bytes to allocate. It returns a pointer to the memory that it allocated, or 0 if it was unable to allocate the memory. The `free` function takes a `void*` that is a copy of a pointer that was returned from `malloc` and returns the associated memory to the system. Calling `free(0)` has no effect.

A simple way to write `operator new` and `operator delete` is as follows:

```
void *operator new(size_t size) {
    if (void *mem = malloc(size))
        return mem;
    else
        throw bad_alloc();
}
void operator delete(void *mem) noexcept { free(mem); }
```

and similarly for the other versions of `operator new` and `operator delete`.

EXERCISES SECTION 19.1.1

Exercise 19.1: Write your own `operator new(size_t)` function using `malloc` and use `free` to write the `operator delete(void*)` function.

Exercise 19.2: By default, the allocator class uses `operator new` to obtain storage and `operator delete` to free it. Recompile and rerun your `StrVec` programs (§ 13.5, p. 526) using your versions of the functions from the previous exercise.

19.1.2 Placement new Expressions

Although the `operator new` and `operator delete` functions are intended to be used by `new` expressions, they are ordinary functions in the library. As a result, ordinary code can call these functions directly.

In earlier versions of the language—before the allocator (§ 12.2.2, p. 481) class was part of the library—applications that wanted to separate allocation from initialization did so by calling `operator new` and `operator delete`. These functions behave analogously to the `allocate` and `deallocate` members of `allocator`. Like those members, `operator new` and `operator delete` functions allocate and deallocate memory but do not construct or destroy objects.

Differently from an allocator, there is no `construct` function we can call to construct objects in memory allocated by `operator new`. Instead, we use the **placement new** form of `new` (§ 12.1.2, p. 460) to construct an object. As we've seen, this form of `new` provides extra information to the allocation function. We can use placement `new` to pass an address, in which case the placement `new` expression has the form

```
new (place_address) type
new (place_address) type (initializers)
new (place_address) type [size]
new (place_address) type [size] { braced_initializer_list }
```

where *place_address* must be a pointer and the *initializers* provide (a possibly empty) comma-separated list of initializers to use to construct the newly allocated object.

When called with an address and no other arguments, placement `new` uses `operator new(size_t, void*)` to “allocate” its memory. This is the version of `operator new` that we are not allowed to redefine (§ 19.1.1, p. 822). This function does *not* allocate any memory; it simply returns its pointer argument. The overall `new` expression then finishes its work by initializing an object at the given address. In effect, placement `new` allows us to construct an object at a specific, preallocated memory address.



When passed a single argument that is a pointer, a placement `new` expression constructs an object but does not allocate memory.

Although in many ways using placement `new` is analogous to the `construct` member of an allocator, there is one important difference. The pointer that we pass to construct must point to space allocated by the same allocator object. The pointer that we pass to placement `new` need not point to memory allocated by `operator new`. Indeed, as we'll see in § 19.6 (p. 851), the pointer passed to a placement `new` expression need not even refer to dynamic memory.

Explicit Destructor Invocation

Just as placement `new` is analogous to using `allocate`, an explicit call to a destructor is analogous to calling `destroy`. We call a destructor the same way we call any other member function on an object or through a pointer or reference to an object:

```
string *sp = new string("a value"); // allocate and initialize a string
sp->~string();
```

Here we invoke a destructor directly. The arrow operator dereferences the pointer `sp` to obtain the object to which `sp` points. We then call the destructor, which is the name of the type preceded by a tilde (`~`).

Like calling `destroy`, calling a destructor cleans up the given object but does not free the space in which that object resides. We can reuse the space if desired.



Calling a destructor destroys an object but does not free the memory.

19.2 Run-Time Type Identification

Run-time type identification (RTTI) is provided through two operators:

- The `typeid` operator, which returns the type of a given expression
- The `dynamic_cast` operator, which safely converts a pointer or reference to a base type into a pointer or reference to a derived type

When applied to pointers or references to types that have virtual functions, these operators use the dynamic type (§ 15.2.3, p. 601) of the object to which the pointer or reference is bound.

These operators are useful when we have a derived operation that we want to perform through a pointer or reference to a base-class object and it is not possible to make that operation a virtual function. Ordinarily, we should use virtual functions if we can. When the operation is virtual, the compiler automatically selects the right function according to the dynamic type of the object.

However, it is not always possible to define a virtual. If we cannot use a virtual, we can use one of the RTTI operators. On the other hand, using these operators is more error-prone than using virtual member functions: The programmer must *know* to which type the object should be cast and must check that the cast was performed successfully.



RTTI should be used with caution. When possible, it is better to define a virtual function rather than to take over managing the types directly.

19.2.1 The `dynamic_cast` Operator

A `dynamic_cast` has the following form:

```
dynamic_cast<type*>(e)
dynamic_cast<type&>(e)
dynamic_cast<type&&>(e)
```

where *type* must be a class type and (ordinarily) names a class that has virtual functions. In the first case, *e* must be a valid pointer (§ 2.3.2, p. 52); in the second, *e* must be an lvalue; and in the third, *e* must not be an lvalue.

In all cases, the type of *e* must be either a class type that is publicly derived from the target *type*, a `public` base class of the target *type*, or the same as the target *type*. If *e* has one of these types, then the cast will succeed. Otherwise, the cast fails. If a `dynamic_cast` to a pointer type fails, the result is 0. If a `dynamic_cast` to a reference type fails, the operator throws an exception of type `bad_cast`.

Pointer-Type `dynamic_cast`s

As a simple example, assume that `Base` is a class with at least one virtual function and that class `Derived` is publicly derived from `Base`. If we have a pointer to `Base` named `bp`, we can cast it, at run time, to a pointer to `Derived` as follows:

```

if (Derived *dp = dynamic_cast<Derived*>(bp))
{
    // use the Derived object to which dp points
} else { // bp points at a Base object
    // use the Base object to which bp points
}

```

If `bp` points to a Derived object, then the cast will initialize `dp` to point to the Derived object to which `bp` points. In this case, it is safe for the code inside the `if` to use Derived operations. Otherwise, the result of the cast is 0. If `dp` is 0, the condition in the `if` fails. In this case, the `else` clause does processing appropriate to Base instead.



We can do a `dynamic_cast` on a null pointer; the result is a null pointer of the requested type.

It is worth noting that we defined `dp` inside the condition. By defining the variable in a condition, we do the cast and corresponding check as a single operation. Moreover, the pointer `dp` is not accessible outside the `if`. If the cast fails, then the unbound pointer is not available for use in subsequent code where we might forget to check whether the cast succeeded.



Performing a `dynamic_cast` in a condition ensures that the cast and test of its result are done in a single expression.

Reference-Type `dynamic_cast`s

A `dynamic_cast` to a reference type differs from a `dynamic_cast` to a pointer type in how it signals that an error occurred. Because there is no such thing as a null reference, it is not possible to use the same error-reporting strategy for references that is used for pointers. When a cast to a reference type fails, the cast throws a `std::bad_cast` exception, which is defined in the `typeinfo` library header.

We can rewrite the previous example to use references as follows:

```

void f(const Base &b)
{
    try {
        const Derived &d = dynamic_cast<const Derived&>(b);
        // use the Derived object to which b referred
    } catch (bad_cast) {
        // handle the fact that the cast failed
    }
}

```

19.2.2 The `typeid` Operator

The second operator provided for RTTI is the **`typeid` operator**. The `typeid` operator allows a program to ask of an expression: What type is your object?

EXERCISES SECTION 19.2.1

Exercise 19.3: Given the following class hierarchy in which each class defines a public default constructor and virtual destructor:

```
class A { /* ... */ };
class B : public A { /* ... */ };
class C : public B { /* ... */ };
class D : public B, public A { /* ... */ };
```

which, if any, of the following `dynamic_cast`s fail?

- (a) `A *pa = new C;`
`B *pb = dynamic_cast< B* >(pa);`
- (b) `B *pb = new B;`
`C *pc = dynamic_cast< C* >(pb);`
- (c) `A *pa = new D;`
`B *pb = dynamic_cast< B* >(pa);`

Exercise 19.4: Using the classes defined in the first exercise, rewrite the following code to convert the expression `*pa` to the type `C&`:

```
if (C *pc = dynamic_cast< C* >(pa))
    // use C's members
} else {
    // use A's members
}
```

Exercise 19.5: When should you use a `dynamic_cast` instead of a virtual function?

A `typeid` expression has the form `typeid(e)` where `e` is any expression or a type name. The result of a `typeid` operation is a reference to a `const` object of a library type named `type_info`, or a type publicly derived from `type_info`. § 19.2.4 (p. 831) covers this type in more detail. The `type_info` class is defined in the `typeinfo` header.

The `typeid` operator can be used with expressions of any type. As usual, top-level `const` (§ 2.4.3, p. 63) is ignored, and if the expression is a reference, `typeid` returns the type to which the reference refers. When applied to an array or function, however, the standard conversion to pointer (§ 4.11.2, p. 161) is not done. That is, if we take `typeid(a)` and `a` is an array, the result describes an array type, not a pointer type.

When the operand is not of class type or is a class without virtual functions, then the `typeid` operator indicates the static type of the operand. When the operand is an lvalue of a class type that defines at least one virtual function, then the type is evaluated at run time.

Using the `typeid` Operator

Ordinarily, we use `typeid` to compare the types of two expressions or to compare the type of an expression to a specified type:

```

Derived *dp = new Derived;
Base *bp = dp;  // both pointers point to a Derived object
// compare the type of two objects at run time
if (typeid(*bp) == typeid(*dp)) {
    // bp and dp point to objects of the same type
}
// test whether the run-time type is a specific type
if (typeid(*bp) == typeid(Derived)) {
    // bp actually points to a Derived
}

```

In the first `if`, we compare the dynamic types of the objects to which `bp` and `dp` point. If both point to the same type, then the condition succeeds. Similarly, the second `if` succeeds if `bp` currently points to a `Derived` object.

Note that the operands to the `typeid` are objects—we used `*bp`, not `bp`:

```

// test always fails: the type of bp is pointer to Base
if (typeid(bp) == typeid(Derived)) {
    // code never executed
}

```

This condition compares the type `Base*` to type `Derived`. Although the pointer *points* at an object of class type that has virtual functions, the pointer *itself* is not a class-type object. The type `Base*` can be, and is, evaluated at compile time. That type is unequal to `Derived`, so the condition will always fail *regardless of the type of the object to which* `bp` *points*.



WARNING

The `typeid` of a pointer (as opposed to the object to which the pointer points) returns the static, compile-time type of the pointer.

Whether `typeid` requires a run-time check determines whether the expression is evaluated. The compiler evaluates the expression only if the type has virtual functions. If the type has no virtuals, then `typeid` returns the static type of the expression; the compiler knows the static type without evaluating the expression.

If the dynamic type of the expression might differ from the static type, then the expression must be evaluated (at run time) to determine the resulting type. The distinction matters when we evaluate `typeid(*p)`. If `p` is a pointer to a type that does not have virtual functions, then `p` does not need to be a valid pointer. Otherwise, `*p` is evaluated at run time, in which case `p` must be a valid pointer. If `p` is a null pointer, then `typeid(*p)` throws a `bad_typeid` exception.

19.2.3 Using RTTI

As an example of when RTTI might be useful, consider a class hierarchy for which we'd like to implement the equality operator (§ 14.3.1, p. 561). Two objects are equal if they have the same type and same value for a given set of their data members. Each derived type may add its own data, which we will want to include when we test for equality.

EXERCISES SECTION 19.2.2

Exercise 19.6: Write an expression to dynamically cast a pointer to a `Query_base` to a pointer to an `AndQuery` (§ 15.9.1, p. 636). Test the cast by using objects of `AndQuery` and of another query type. Print a statement indicating whether the cast works and be sure that the output matches your expectations.

Exercise 19.7: Write the same cast, but cast a `Query_base` object to a reference to `AndQuery`. Repeat the test to ensure that your cast works correctly.

Exercise 19.8: Write a `typeid` expression to see whether two `Query_base` pointers point to the same type. Now check whether that type is an `AndQuery`.

We might think we could solve this problem by defining a set of virtual functions that would perform the equality test at each level in the hierarchy. Given those virtuals, we would define a single equality operator that operates on references to the base type. That operator could delegate its work to a virtual `equal` operation that would do the real work.

Unfortunately, this strategy doesn't quite work. Virtual functions must have the same parameter type(s) in both the base and derived classes (§ 15.3, p. 605). If we wanted to define a virtual `equal` function, that function must have a parameter that is a reference to the base class. If the parameter is a reference to base, the `equal` function could use only members from the base class. `equal` would have no way to compare members that are in the derived class but not in the base.

We can write our equality operation by realizing that the equality operator ought to return `false` if we attempt to compare objects of differing type. For example, if we try to compare a object of the base-class type with an object of a derived type, the `==` operator should return `false`.

Given this observation, we can now see that we can use RTTI to solve our problem. We'll define an equality operator whose parameters are references to the base-class type. The equality operator will use `typeid` to verify that the operands have the same type. If the operands differ, the `==` will return `false`. Otherwise, it will call a virtual `equal` function. Each class will define `equal` to compare the data elements of its own type. These operators will take a `Base&` parameter but will cast the operand to its own type before doing the comparison.

The Class Hierarchy

To make the concept a bit more concrete, we'll define the following classes:

```
class Base {
    friend bool operator==(const Base&, const Base&);
public:
    // interface members for Base
protected:
    virtual bool equal(const Base&) const;
    // data and other implementation members of Base
};
```

```

class Derived: public Base {
public:
    // other interface members for Derived
protected:
    bool equal(const Base&) const;
    // data and other implementation members of Derived
};

```

A Type-Sensitive Equality Operator

Next let's look at how we might define the overall equality operator:

```

bool operator==(const Base &lhs, const Base &rhs)
{
    // returns false if typeid's are different; otherwise makes a virtual call to equal
    return typeid(lhs) == typeid(rhs) && lhs.equal(rhs);
}

```

This operator returns `false` if the operands are different types. If they are the same type, then it delegates the real work of comparing the operands to the (virtual) `equal` function. If the operands are `Base` objects, then `Base::equal` will be called. If they are `Derived` objects, `Derived::equal` is called.

The Virtual `equal` Functions

Each class in the hierarchy must define its own version of `equal`. All of the functions in the derived classes will start the same way: They'll cast their argument to the type of the class itself:

```

bool Derived::equal(const Base &rhs) const
{
    // we know the types are equal, so the cast won't throw
    auto r = dynamic_cast<const Derived&>(rhs);
    // do the work to compare two Derived objects and return the result
}

```

The cast should always succeed—after all, the function is called from the equality operator only after testing that the two operands are the same type. However, the cast is necessary so that the function can access the derived members of the right-hand operand.

The Base-Class `equal` Function

This operation is a bit simpler than the others:

```

bool Base::equal(const Base &rhs) const
{
    // do whatever is required to compare to Base objects
}

```

There is no need to cast the parameter before using it. Both `*this` and the parameter are `Base` objects, so all the operations available for this object are also defined for the parameter type.

19.2.4 The `type_info` Class

The exact definition of the `type_info` class varies by compiler. However, the standard guarantees that the class will be defined in the `typeinfo` header and that the class will provide at least the operations listed in Table 19.1.

The class also provides a public virtual destructor, because it is intended to serve as a base class. When a compiler wants to provide additional type information, it normally does so in a class derived from `type_info`.

Table 19.1: Operations on `type_info`

<code>t1 == t2</code>	Returns true if the <code>type_info</code> objects <code>t1</code> and <code>t2</code> refer to the same type, false otherwise.
<code>t1 != t2</code>	Returns true if the <code>type_info</code> objects <code>t1</code> and <code>t2</code> refer to different types, false otherwise.
<code>t.name()</code>	Returns a C-style character string that is a printable version of the type name. Type names are generated in a system-dependent way.
<code>t1.before(t2)</code>	Returns a <code>bool</code> that indicates whether <code>t1</code> comes before <code>t2</code> . The ordering imposed by <code>before</code> is compiler dependent.

There is no `type_info` default constructor, and the copy and move constructors and the assignment operators are all defined as deleted (§ 13.1.6, p. 507). Therefore, we cannot define, copy, or assign objects of type `type_info`. The only way to create a `type_info` object is through the `typeid` operator.

The `name` member function returns a C-style character string for the name of the type represented by the `type_info` object. The value used for a given type depends on the compiler and in particular is not required to match the type names as used in a program. The only guarantee we have about the return from `name` is that it returns a unique string for each type. For example:

```
int arr[10];
Derived d;
Base *p = &d;

cout << typeid(42).name() << ", "
     << typeid(arr).name() << ", "
     << typeid(Sales_data).name() << ", "
     << typeid(std::string).name() << ", "
     << typeid(p).name() << ", "
     << typeid(*p).name() << endl;
```

This program, when executed on our machine, generates the following output:

```
i, A10_i, 10Sales_data, Ss, P4Base, 7Derived
```



The `type_info` class varies by compiler. Some compilers provide additional member functions that provide additional information about types used in a program. You should consult the reference manual for your compiler to understand the exact `type_info` support provided.

EXERCISES SECTION 19.2.4

Exercise 19.9: Write a program similar to the last one in this section to print the names your compiler uses for common type names. If your compiler gives output similar to ours, write a function that will translate those strings to more human-friendly form.

Exercise 19.10: Given the following class hierarchy in which each class defines a public default constructor and virtual destructor, which type name do the following statements print?

```
class A { /* ... */ };
class B : public A { /* ... */ };
class C : public B { /* ... */ };

(a) A *pa = new C;
    cout << typeid(pa).name() << endl;

(b) C cobj;
    A& ra = cobj;
    cout << typeid(&ra).name() << endl;

(c) B *px = new B;
    A& ra = *px;
    cout << typeid(ra).name() << endl;
```

19.3 Enumerations

Enumerations let us group together sets of integral constants. Like classes, each enumeration defines a new type. Enumerations are literal types (§ 7.5.6, p. 299).

C++ has two kinds of enumerations: **scoped** and **unscoped**. The new standard introduced **scoped enumerations**. We define a scoped enumeration using the keywords `enum class` (or, equivalently, `enum struct`), followed by the enumeration name and a comma-separated list of **enumerators** enclosed in curly braces. A semicolon follows the close curly:

C++
11

```
enum class open_modes {input, output, append};
```

Here we defined an enumeration type named `open_modes` that has three enumerators: `input`, `output`, and `append`.

We define an **unscoped enumeration** by omitting the `class` (or `struct`) keyword. The enumeration name is optional in an unscoped enum:

```
enum color {red, yellow, green};           // unscoped enumeration
// unnamed, unscoped enum
enum {floatPrec = 6, doublePrec = 10, double_doublePrec = 10};
```

If the enum is unnamed, we may define objects of that type only as part of the enum definition. As with a class definition, we can provide a comma-separated list of declarators between the close curly and the semicolon that ends the enum definition (§ 2.6.1, p. 73).

Enumerators

The names of the enumerators in a scoped enumeration follow normal scoping rules and are inaccessible outside the scope of the enumeration. The enumerator names in an unscoped enumeration are placed into the same scope as the enumeration itself:

```
enum color {red, yellow, green};           // unscoped enumeration
enum stoplight {red, yellow, green};      // error: redefines enumerators
enum class peppers {red, yellow, green};   // ok: enumerators are hidden
color eyes = green;                       // ok: enumerators are in scope for an unscoped enumeration
peppers p = green;                        // error: enumerators from peppers are not in scope
// color::green is in scope but has the wrong type
color hair = color::red;                  // ok: we can explicitly access the enumerators
peppers p2 = peppers::red;                // ok: using red from peppers
```

By default, enumerator values start at 0 and each enumerator has a value 1 greater than the preceding one. However, we can also supply initializers for one or more enumerators:

```
enum class intTypes {
    charTyp = 8, shortTyp = 16, intTyp = 16,
    longTyp = 32, long_longTyp = 64
};
```

As we see with the enumerators for `intTyp` and `shortTyp`, an enumerator value need not be unique. When we omit an initializer, the enumerator has a value 1 greater than the preceding enumerator.

Enumerators are `const` and, if initialized, their initializers must be constant expressions (§ 2.4.4, p. 65). Consequently, each enumerator is itself a constant expression. Because the enumerators are constant expressions, we can use them where a constant expression is required. For example, we can define `constexpr` variables of enumeration type:

```
constexpr intTypes charbits = intTypes::charTyp;
```

Similarly, we can use an `enum` as the expression in a `switch` statement and use the value of its enumerators as the `case` labels (§ 5.3.2, p. 178). For the same reason, we can also use an enumeration type as a nontype template parameter (§ 16.1.1, p. 654), and can initialize class `static` data members of enumeration type inside the class definition (§ 7.6, p. 302).

Like Classes, Enumerations Define New Types

So long as the `enum` is named, we can define and initialize objects of that type. An `enum` object may be initialized or assigned only by one of its enumerators or by another object of the same `enum` type:

```
open_modes om = 2;           // error: 2 is not of type open_modes
om = open_modes::input;      // ok: input is an enumerator of open_modes
```

Objects or enumerators of an unscoped enumeration type are automatically converted to an integral type. As a result, they can be used where an integral value is required:

```
int i = color::red;    // ok: unscoped enumerator implicitly converted to int
int j = peppers::red; // error: scoped enumerations are not implicitly converted
```

Specifying the Size of an enum

Although each enum defines a unique type, it is represented by one of the built-in integral types. Under the new standard, we may specify that type by following the enum name with a colon and the name of the type we want to use:

**C++
11**

```
enum intValues : unsigned long long {
    charTyp = 255, shortTyp = 65535, intTyp = 65535,
    longTyp = 4294967295UL,
    long_longTyp = 18446744073709551615ULL
};
```

If we do not specify the underlying type, then by default scoped enums have `int` as the underlying type. There is no default for unscoped enums; all we know is that the underlying type is large enough to hold the enumerator values. When the underlying type is specified (including implicitly specified for a scoped enum), it is an error for an enumerator to have a value that is too large to fit in that type.

Being able to specify the underlying type of an enum lets us control the type used across different implementations. We can be confident that our program compiled under one implementation will generate the same code when we compile it on another.

Forward Declarations for Enumerations

**C++
11**

Under the new standard, we can forward declare an enum. An enum forward declaration must specify (implicitly or explicitly) the underlying size of the enum:

```
// forward declaration of unscoped enum named intValues
enum intValues : unsigned long long; // unscoped, must specify a type
enum class open_modes; // scoped enums can use int by default
```

Because there is no default size for an unscoped enum, every declaration must include the size of that enum. We can declare a scoped enum without specifying a size, in which case the size is implicitly defined as `int`.

As with any declaration, all the declarations and the definition of a given enum must match one another. In the case of enums, this requirement means that the size of the enum must be the same across all declarations and the enum definition. Moreover, we cannot declare a name as an unscoped enum in one context and redeclare it as a scoped enum later:

```
// error: declarations and definition must agree whether the enum is scoped or unscoped
enum class intValues;
enum intValues; // error: intValues previously declared as scoped enum
enum intValues : long; // error: intValues previously declared as int
```

Parameter Matching and Enumerations

Because an object of enum type may be initialized only by another object of that enum type or by one of its enumerators (§ 19.3, p. 833), an integral value that happens to have the same value as an enumerator cannot be used to call a function expecting an enum argument:

```
// unscoped enumeration; the underlying type is machine dependent
enum Tokens { INLINE = 128, VIRTUAL = 129 };

void ff(Tokens);
void ff(int);

int main() {
    Tokens curTok = INLINE;
    ff(128);      // exactly matches ff(int)
    ff(INLINE);   // exactly matches ff(Tokens)
    ff(curTok);   // exactly matches ff(Tokens)
    return 0;
}
```

Although we cannot pass an integral value to an enum parameter, we can pass an object or enumerator of an unscoped enumeration to a parameter of integral type. When we do so, the enum value promotes to `int` or to a larger integral type. The actual promotion type depends on the underlying type of the enumeration:

```
void newf(unsigned char);
void newf(int);
unsigned char uc = VIRTUAL;
newf(VIRTUAL); // calls newf(int)
newf(uc);      // calls newf(unsigned char)
```

The enum `Tokens` has only two enumerators, the larger of which has the value 129. That value can be represented by the type `unsigned char`, and many compilers will use `unsigned char` as the underlying type for `Tokens`. Regardless of its underlying type, objects and the enumerators of `Tokens` are promoted to `int`. Enumerators and values of an enum type are not promoted to `unsigned char`, even if the values of the enumerators would fit.

19.4 Pointer to Class Member

A **pointer to member** is a pointer that can point to a nonstatic member of a class. Normally a pointer points to an object, but a pointer to member identifies a member of a class, not an object of that class. static class members are not part of any object, so no special syntax is needed to point to a static member. Pointers to static members are ordinary pointers.

The type of a pointer to member embodies both the type of a class and the type of a member of that class. We initialize such pointers to point to a specific member of a class without identifying an object to which that member belongs. When we use a pointer to member, we supply the object whose member we wish to use.

To explain pointers to members, we'll use a version of the `Screen` class from § 7.3.1 (p. 271):

```
class Screen {
public:
    typedef std::string::size_type pos;
    char get_cursor() const { return contents[cursor]; }
    char get() const;
    char get(pos ht, pos wd) const;
private:
    std::string contents;
    pos cursor;
    pos height, width;
};
```

19.4.1 Pointers to Data Members

As with any pointer, we declare a pointer to member using a `*` to indicate that the name we're declaring is a pointer. Unlike ordinary pointers, a pointer to member also incorporates the class that contains the member. Hence, we must precede the `*` with `classname::` to indicate that the pointer we are defining can point to a member of `classname`. For example:

```
// pdata can point to a string member of a const (or nonconst) Screen object
const string Screen::*pdata;
```

declares that `pdata` is a “pointer to a member of class `Screen` that has type `const string`.” The data members in a `const` object are themselves `const`. By making our pointer a pointer to `const string` member, we say that we can use `pdata` to point to a member of any `Screen` object, `const` or not. In exchange we can use `pdata` to read, but not write to, the member to which it points.

When we initialize (or assign to) a pointer to member, we say to which member it points. For example, we can make `pdata` point to the `contents` member of an unspecified `Screen` object as follows:

```
pdata = &Screen::contents;
```

Here, we apply the address-of operator `&` not to an object in memory but to a member of the class `Screen`.

Of course, under the new standard, the easiest way to declare a pointer to member is to use `auto` or `decltype`:

```
auto pdata = &Screen::contents;
```

Using a Pointer to Data Member

It is essential to understand that when we initialize or assign a pointer to member, that pointer does not yet point to any data. It identifies a specific member but not the object that contains that member. We supply the object when we dereference the pointer to member.

Analogous to the member access operators, `.` and `->`, there are two pointer-to-member access operators, `.*` and `->*`, that let us supply an object and dereference the pointer to fetch a member of that object:

```
Screen myScreen, *pScreen = &myScreen;
//  .* dereferences pdata to fetch the contents member from the object myScreen
auto s = myScreen.*pdata;
//  ->* dereferences pdata to fetch contents from the object to which pScreen points
s = pScreen->*pdata;
```

Conceptually, these operators perform two actions: They dereference the pointer to member to get the member that we want; then, like the member access operators, they fetch that member from an object (`.*`) or through a pointer (`->*`).

A Function Returning a Pointer to Data Member

Normal access controls apply to pointers to members. For example, the `contents` member of `Screen` is `private`. As a result, the use of `pdata` above must have been inside a member or friend of class `Screen` or it would be an error.

Because data members are typically `private`, we normally can't get a pointer to data member directly. Instead, if a class like `Screen` wanted to allow access to its `contents` member, it would define a function to return a pointer to that member:

```
class Screen {
public:
    //  data is a static member that returns a pointer to member
    static const std::string Screen::*data()
        { return &Screen::contents; }
    //  other members as before
};
```

Here we've added a static member to class `Screen` that returns a pointer to the `contents` member of a `Screen`. The return type of this function is the same type as our original `pdata` pointer. Reading the return type from right to left, we see that `data` returns a pointer to a member of class `Screen` that is a `string` that is `const`. The body of the function applies the address-of operator to the `contents` member, so the function returns a pointer to the `contents` member of `Screen`.

When we call `data`, we get a pointer to member:

```
//  data() returns a pointer to the contents member of class Screen
const string Screen::*pdata = Screen::data();
```

As before, `pdata` points to a member of class `Screen` but not to actual data. To use `pdata`, we must bind it to an object of type `Screen`

```
//  fetch the contents of the object named myScreen
auto s = myScreen.*pdata;
```

EXERCISES SECTION 19.4.1

Exercise 19.11: What is the difference between an ordinary data pointer and a pointer to a data member?

Exercise 19.12: Define a pointer to member that can point to the `cursor` member of class `Screen`. Fetch the value of `Screen::cursor` through that pointer.

Exercise 19.13: Define the type that can represent a pointer to the `bookNo` member of the `Sales_data` class.

19.4.2 Pointers to Member Functions

We can also define a pointer that can point to a member function of a class. As with pointers to data members, the easiest way to form a pointer to member function is to use `auto` to deduce the type for us:

```
// pmf is a pointer that can point to a Screen member function that is const
// that returns a char and takes no arguments
auto pmf = &Screen::get_cursor;
```

Like a pointer to data member, a pointer to a function member is declared using `classname::*`. Like any other function pointer (§ 6.7, p. 247), a pointer to member function specifies the return type and parameter list of the type of function to which this pointer can point. If the member function is a `const` member (§ 7.1.2, p. 258) or a reference member (§ 13.6.3, p. 546), we must include the `const` or reference qualifier as well.

As with normal function pointers, if the member is overloaded, we must distinguish which function we want by declaring the type explicitly (§ 6.7, p. 248). For example, we can declare a pointer to the two-parameter version of `get` as

```
char (Screen::*pmf2)(Screen::pos, Screen::pos) const;
pmf2 = &Screen::get;
```

The parentheses around `Screen::*` in this declaration are essential due to precedence. Without the parentheses, the compiler treats the following as an (invalid) function declaration:

```
// error: nonmember function p cannot have a const qualifier
char Screen::*p(Screen::pos, Screen::pos) const;
```

This declaration tries to define an ordinary function named `p` that returns a pointer to a member of class `Screen` that has type `char`. Because it declares an ordinary function, the declaration can't be followed by a `const` qualifier.

Unlike ordinary function pointers, there is no automatic conversion between a member function and a pointer to that member:

```
// pmf points to a Screen member that takes no arguments and returns char
pmf = &Screen::get; // must explicitly use the address-of operator
pmf = Screen::get; // error: no conversion to pointer for member functions
```

Using a Pointer to Member Function

As when we use a pointer to a data member, we use the `.*` or `->*` operators to call a member function through a pointer to member:

```
Screen myScreen, *pScreen = &myScreen;
// call the function to which pmf points on the object to which pScreen points
char c1 = (pScreen->*pmf)();
// passes the arguments 0, 0 to the two-parameter version of get on the object myScreen
char c2 = (myScreen.*pmf2)(0, 0);
```

The calls `(myScreen->*pmf)()` and `(pScreen.*pmf2)(0, 0)` require the parentheses because the precedence of the call operator is higher than the precedence of the pointer to member operators.

Without the parentheses,

```
myScreen.*pmf()
```

would be interpreted to mean

```
myScreen.*(pmf())
```

This code says to call the function named `pmf` and use its return value as the operand of the pointer-to-member operator `(.*)`. However, `pmf` is not a function, so this code is in error.



Because of the relative precedence of the call operator, declarations of pointers to member functions and calls through such pointers must use parentheses: `(C::*p)(parms)` and `(obj.*p)(args)`.

Using Type Aliases for Member Pointers

Type aliases or `typedefs` (§ 2.5.1, p. 67) make pointers to members considerably easier to read. For example, the following type alias defines `Action` as an alternative name for the type of the two-parameter version of `get`:

```
// Action is a type that can point to a member function of Screen
// that returns a char and takes two pos arguments
using Action =
char (Screen::*)(Screen::pos, Screen::pos) const;
```

`Action` is another name for the type “pointer to a `const` member function of class `Screen` taking two parameters of type `pos` and returning `char`.” Using this alias, we can simplify the definition of a pointer to `get` as follows:

```
Action get = &Screen::get; // get points to the get member of Screen
```

As with any other function pointer, we can use a pointer-to-member function type as the return type or as a parameter type in a function. Like any other parameter, a pointer-to-member parameter can have a default argument:

```
// action takes a reference to a Screen and a pointer to a Screen member function
Screen& action(Screen&, Action = &Screen::get);
```

action is a function taking two parameters, which are a reference to a Screen object and a pointer to a member function of class Screen that takes two pos parameters and returns a char. We can call action by passing it either a pointer or the address of an appropriate member function in Screen:

```
Screen myScreen;
// equivalent calls:
action(myScreen);           // uses the default argument
action(myScreen, get);      // uses the variable get that we previously defined
action(myScreen, &Screen::get); // passes the address explicitly
```



Type aliases make code that uses pointers to members much easier to read and write.

Pointer-to-Member Function Tables

One common use for function pointers and for pointers to member functions is to store them in a function table (§ 14.8.3, p. 577). For a class that has several members of the same type, such a table can be used to select one from the set of these members. Let's assume that our Screen class is extended to contain several member functions, each of which moves the cursor in a particular direction:

```
class Screen {
public:
    // other interface and implementation members as before
    Screen& home();           // cursor movement functions
    Screen& forward();
    Screen& back();
    Screen& up();
    Screen& down();
};
```

Each of these new functions takes no parameters and returns a reference to the Screen on which it was invoked.

We might want to define a move function that can call any one of these functions and perform the indicated action. To support this new function, we'll add a static member to Screen that will be an array of pointers to the cursor movement functions:

```
class Screen {
public:
    // other interface and implementation members as before
    // Action is a pointer that can be assigned any of the cursor movement members
    using Action = Screen& (Screen::*)();
    // specify which direction to move; enum see § 19.3 (p. 832)
    enum Directions { HOME, FORWARD, BACK, UP, DOWN };
    Screen& move(Directions);
private:
    static Action Menu[];           // function table
};
```

The array named `Menu` will hold pointers to each of the cursor movement functions. Those functions will be stored at the offsets corresponding to the enumerators in `Directions`. The move function takes an enumerator and calls the appropriate function:

```
Screen& Screen::move(Directions cm)
{
    // run the element indexed by cm on this object
    return (this->*Menu[cm])(); // Menu[cm] points to a member function
}
```

The call inside `move` is evaluated as follows: The `Menu` element indexed by `cm` is fetched. That element is a pointer to a member function of the `Screen` class. We call the member function to which that element points on behalf of the object to which `this` points.

When we call `move`, we pass it an enumerator that indicates which direction to move the cursor:

```
Screen myScreen;
myScreen.move(Screen::HOME); // invokes myScreen.home
myScreen.move(Screen::DOWN); // invokes myScreen.down
```

What's left is to define and initialize the table itself:

```
Screen::Action Screen::Menu[] = { &Screen::home,
                                   &Screen::forward,
                                   &Screen::back,
                                   &Screen::up,
                                   &Screen::down,
                                   };
```

EXERCISES SECTION 19.4.2

Exercise 19.14: Is the following code legal? If so, what does it do? If not, why?

```
auto pmf = &Screen::get_cursor;
pmf = &Screen::get;
```

Exercise 19.15: What is the difference between an ordinary function pointer and a pointer to a member function?

Exercise 19.16: Write a type alias that is a synonym for a pointer that can point to the `avg_price` member of `Sales_data`.

Exercise 19.17: Define a type alias for each distinct `Screen` member function type.

19.4.3 Using Member Functions as Callable Objects

As we've seen, to make a call through a pointer to member function, we must use the `.*` or `->*` operators to bind the pointer to a specific object. As a result,

unlike ordinary function pointers, a pointer to member is *not* a callable object; these pointers do not support the function-call operator (§ 10.3.2, p. 388).

Because a pointer to member is not a callable object, we cannot directly pass a pointer to a member function to an algorithm. As an example, if we wanted to find the first empty string in a vector of strings, the obvious call won't work:

```
auto fp = &string::empty; // fp points to the string empty function
// error: must use .* or ->* to call a pointer to member
find_if(svec.begin(), svec.end(), fp);
```

The `find_if` algorithm expects a callable object, but we've supplied `fp`, which is a pointer to a member function. This call won't compile, because the code inside `find_if` executes a statement something like

```
// check whether the given predicate applied to the current element yields true
if (fp(*it)) // error: must use ->* to call through a pointer to member
```

which attempts to call the object it was passed.

Using function to Generate a Callable

One way to obtain a callable from a pointer to member function is by using the library function template (§ 14.8.3, p. 577):

```
function<bool (const string&)> fcn = &string::empty;
find_if(svec.begin(), svec.end(), fcn);
```

Here we tell `function` that `empty` is a function that can be called with a `string` and returns a `bool`. Ordinarily, the object on which a member function executes is passed to the implicit `this` parameter. When we want to use `function` to generate a callable for a member function, we have to “translate” the code to make that implicit parameter explicit.

When a function object holds a pointer to a member function, the `function` class knows that it must use the appropriate pointer-to-member operator to make the call. That is, we can imagine that `find_if` will have code something like

```
// assuming it is the iterator inside find_if, so *it is an object in the given range
if (fcn(*it)) // assuming fcn is the name of the callable inside find_if
```

which `function` will execute using the proper pointer-to-member operator. In essence, the `function` class will transform this call into something like

```
// assuming it is the iterator inside find_if, so *it is an object in the given range
if (((*it).*p)()) // assuming p is the pointer to member function inside fcn
```

When we define a function object, we must specify the function type that is the signature of the callable objects that object can represent. When the callable is a member function, the signature's first parameter must represent the (normally implicit) object on which the member will be run. The signature we give to `function` must specify whether the object will be passed as a pointer or a reference.

When we defined `fcn`, we knew that we wanted to call `find_if` on a sequence of `string` objects. Hence, we asked `function` to generate a callable that took `string` objects. Had our vector held pointers to `string`, we would have told `function` to expect a pointer:

```
vector<string*> pvec;
function<bool (const string*)> fp = &string::empty;
// fp takes a pointer to string and uses the ->* to call empty
find_if(pvec.begin(), pvec.end(), fp);
```

Using `mem_fn` to Generate a Callable

To use `function`, we must supply the call signature of the member we want to call. We can, instead, let the compiler deduce the member's type by using another library facility, `mem_fn`, which, like `function`, is defined in the functional header. Like `function`, `mem_fn` generates a callable object from a pointer to member. Unlike `function`, `mem_fn` will deduce the type of the callable from the type of the pointer to member:

C++
11

```
find_if(svec.begin(), svec.end(), mem_fn(&string::empty));
```

Here we used `mem_fn(&string::empty)` to generate a callable object that takes a `string` argument and returns a `bool`.

The callable generated by `mem_fn` can be called on either an object or a pointer:

```
auto f = mem_fn(&string::empty); // f takes a string or a string*
f(*svec.begin()); // ok: passes a string object; f uses .* to call empty
f(&svec[0]); // ok: passes a pointer to string; f uses -> to call empty
```

Effectively, we can think of `mem_fn` as if it generates a callable with an overloaded function call operator—one that takes a `string*` and the other a `string&`.

Using `bind` to Generate a Callable

For completeness, we can also use `bind` (§ 10.3.4, p. 397) to generate a callable from a member function:

```
// bind each string in the range to the implicit first argument to empty
auto it = find_if(svec.begin(), svec.end(),
                 bind(&string::empty, _1));
```

As with `function`, when we use `bind`, we must make explicit the member function's normally implicit parameter that represents the object on which the member function will operate. Like `mem_fn`, the first argument to the callable generated by `bind` can be either a pointer or a reference to a `string`:

```
auto f = bind(&string::empty, _1);
f(*svec.begin()); // ok: argument is a string f will use .* to call empty
f(&svec[0]); // ok: argument is a pointer to string f will use -> to call empty
```

19.5 Nested Classes

A class can be defined within another class. Such a class is a **nested class**, also referred to as a **nested type**. Nested classes are most often used to define implementation classes, such as the `QueryResult` class we used in our text query example (§ 12.3, p. 484).

EXERCISES SECTION 19.4.3

Exercise 19.18: Write a function that uses `count_if` to count how many empty strings there are in a given vector.

Exercise 19.19: Write a function that takes a `vector<Sales_data>` and finds the first element whose average price is greater than some given amount.

Nested classes are independent classes and are largely unrelated to their enclosing class. In particular, objects of the enclosing and nested classes are independent from each other. An object of the nested type does not have members defined by the enclosing class. Similarly, an object of the enclosing class does not have members defined by the nested class.

The name of a nested class is visible within its enclosing class scope but not outside the class. Like any other nested name, the name of a nested class will not collide with the use of that name in another scope.

A nested class can have the same kinds of members as a nonnested class. Just like any other class, a nested class controls access to its own members using access specifiers. The enclosing class has no special access to the members of a nested class, and the nested class has no special access to members of its enclosing class.

A nested class defines a type member in its enclosing class. As with any other member, the enclosing class determines access to this type. A nested class defined in the `public` part of the enclosing class defines a type that may be used anywhere. A nested class defined in the `protected` section defines a type that is accessible only by the enclosing class, its friends, and its derived classes. A `private` nested class defines a type that is accessible only to the members and friends of the enclosing class.

Declaring a Nested Class

The `TextQuery` class from § 12.3.2 (p. 487) defined a companion class named `QueryResult`. The `QueryResult` class is tightly coupled to our `TextQuery` class. It would make little sense to use `QueryResult` for any other purpose than to represent the results of a query operation on a `TextQuery` object. To reflect this tight coupling, we'll make `QueryResult` a member of `TextQuery`.

```
class TextQuery {
public:
    class QueryResult; // nested class to be defined later
    // other members as in § 12.3.2 (p. 487)
};
```

We need to make only one change to our original `TextQuery` class—we declare our intention to define `QueryResult` as a nested class. Because `QueryResult` is a type member (§ 7.4.1, p. 284), we must declare `QueryResult` before we use it. In particular, we must declare `QueryResult` before we use it as the return type for the `query` member. The remaining members of our original class are unchanged.

Defining a Nested Class outside of the Enclosing Class

Inside `TextQuery` we declared `QueryResult` but did not define it. As with member functions, nested classes must be declared inside the class but can be defined either inside or outside the class.

When we define a nested class outside its enclosing class, we must qualify the name of the nested class by the name of its enclosing class:

```
// we're defining the QueryResult class that is a member of class TextQuery
class TextQuery::QueryResult {
    // in class scope, we don't have to qualify the name of the QueryResult parameters
    friend std::ostream&
        print(std::ostream&, const QueryResult&);
public:
    // no need to define QueryResult::line_no; a nested class can use a member
    // of its enclosing class without needing to qualify the member's name
    QueryResult(std::string,
                std::shared_ptr<std::set<line_no>>,
                std::shared_ptr<std::vector<std::string>>);
    // other members as in § 12.3.2 (p. 487)
};
```

The only change we made compared to our original class is that we no longer define a `line_no` member in `QueryResult`. The members of `QueryResult` can access that name directly from `TextQuery`, so there is no need to define it again.



WARNING

Until the actual definition of a nested class that is defined outside the class body is seen, that class is an incomplete type (§ 7.3.3, p. 278).

Defining the Members of a Nested Class

In this version, we did not define the `QueryResult` constructor inside the class body. To define the constructor, we must indicate that `QueryResult` is nested within the scope of `TextQuery`. We do so by qualifying the nested class name with the name of its enclosing class:

```
// defining the member named QueryResult for the class named QueryResult
// that is nested inside the class TextQuery
TextQuery::QueryResult::QueryResult(string s,
                                     shared_ptr<set<line_no>> p,
                                     shared_ptr<vector<string>> f):
    sought(s), lines(p), file(f) { }
```

Reading the name of the function from right to left, we see that we are defining the constructor for class `QueryResult`, which is nested in the scope of class `TextQuery`. The code itself just stores the given arguments in the data members and has no further work to do.

Nested-Class static Member Definitions

If `QueryResult` had declared a static member, its definition would appear outside the scope of the `TextQuery`. For example, assuming `QueryResult` had a

static member, its definition would look something like

```
// defines an int static member of QueryResult
// which is a class nested inside TextQuery
int TextQuery::QueryResult::static_mem = 1024;
```

Name Lookup in Nested Class Scope

Normal rules apply for name lookup (§ 7.4.1, p. 283) inside a nested class. Of course, because a nested class is a nested scope, the nested class has additional enclosing class scopes to search. This nesting of scopes explains why we didn't define `line_no` inside the nested version of `QueryResult`. Our original `QueryResult` class defined this member so that its own members could avoid having to write `TextQuery::line_no`. Having nested the definition of our results class inside `TextQuery`, we no longer need this typedef. The nested `QueryResult` class can access `line_no` without specifying that `line_no` is defined in `TextQuery`.

As we've seen, a nested class is a type member of its enclosing class. Members of the enclosing class can use the name of a nested class the same way it can use any other type member. Because `QueryResult` is nested inside `TextQuery`, the query member of `TextQuery` can refer to the name `QueryResult` directly:

```
// return type must indicate that QueryResult is now a nested class
TextQuery::QueryResult
TextQuery::query(const string &sought) const
{
    // we'll return a pointer to this set if we don't find sought
    static shared_ptr<set<line_no>> nodata(new set<line_no>);
    // use find and not a subscript to avoid adding words to wm!
    auto loc = wm.find(sought);
    if (loc == wm.end())
        return QueryResult(sought, nodata, file); // not found
    else
        return QueryResult(sought, loc->second, file);
}
```

As usual, the return type is not yet in the scope of the class (§ 7.4, p. 282), so we start by noting that our function returns a `TextQuery::QueryResult` value. However, inside the body of the function, we can refer to `QueryResult` directly, as we do in the return statements.

The Nested and Enclosing Classes Are Independent

Although a nested class is defined in the scope of its enclosing class, it is important to understand that there is no connection between the objects of an enclosing class and objects of its nested classe(s). A nested-type object contains only the members defined inside the nested type. Similarly, an object of the enclosing class has only those members that are defined by the enclosing class. It does not contain the data members of any nested classes.

More concretely, the second return statement in `TextQuery::query`

```
return QueryResult(sought, loc->second, file);
```

uses data members of the `TextQuery` object on which query was run to initialize a `QueryResult` object. We have to use these members to construct the `QueryResult` object we return because a `QueryResult` object does not contain the members of its enclosing class.

EXERCISES SECTION 19.5

Exercise 19.20: Nest your `QueryResult` class inside `TextQuery` and rerun the programs you wrote to use `TextQuery` in § 12.3.2 (p. 490).

19.6 union: A Space-Saving Class

A **union** is a special kind of class. A union may have multiple data members, but at any point in time, only one of the members may have a value. When a value is assigned to one member of the union, all other members become undefined. The amount of storage allocated for a union is at least as much as is needed to contain its largest data member. Like any class, a union defines a new type.

Some, but not all, class features apply equally to unions. A union cannot have a member that is a reference, but it can have members of most other types, including, under the new standard, class types that have constructors or destructors. A union can specify protection labels to make members `public`, `private`, or `protected`. By default, like `structs`, members of a union are `public`.

A union may define member functions, including constructors and destructors. However, a union may not inherit from another class, nor may a union be used as a base class. As a result, a union may not have virtual functions.

Defining a union

unions offer a convenient way to represent a set of mutually exclusive values of different types. As an example, we might have a process that handles different kinds of numeric or character data. That process might define a union to hold these values:

```
// objects of type Token have a single member, which could be of any of the listed types
union Token {
    // members are public by default
    char    cval;
    int     ival;
    double  dval;
};
```

A union is defined starting with the keyword `union`, followed by an (optional) name for the union and a set of member declarations enclosed in curly braces. This code defines a union named `Token` that can hold a value that is either a `char`, an `int`, or a `double`.

Using a union Type

The name of a union is a type name. Like the built-in types, by default unions are uninitialized. We can explicitly initialize a union in the same way that we can explicitly initialize aggregate classes (§ 7.5.5, p. 298) by enclosing the initializer in a pair of curly braces:

```
Token first_token = {'a'}; // initializes the cval member
Token last_token;          // uninitialized Token object
Token *pt = new Token;     // pointer to an uninitialized Token object
```

If an initializer is present, it is used to initialize the first member. Hence, the initialization of `first_token` gives a value to its `cval` member.

The members of an object of union type are accessed using the normal member access operators:

```
last_token.cval = 'z';
pt->ival = 42;
```

Assigning a value to a data member of a union object makes the other data members undefined. As a result, when we use a union, we must always know what type of value is currently stored in the union. Depending on the types of the members, retrieving or assigning to the value stored in the union through the wrong data member can lead to a crash or other incorrect program behavior.

Anonymous unions

An **anonymous union** is an unnamed union that does not include any declarations between the close curly that ends its body and the semicolon that ends the union definition (§ 2.6.1, p. 73). When we define an anonymous union the compiler automatically creates an unnamed object of the newly defined union type:

```
union {                // anonymous union
    char    cval;
    int     ival;
    double  dval;
}; // defines an unnamed object, whose members we can access directly
cval = 'c'; // assigns a new value to the unnamed, anonymous union object
ival = 42;  // that object now holds the value 42
```

The members of an anonymous union are directly accessible in the scope where the anonymous union is defined.



An anonymous union cannot have `private` or `protected` members, nor can an anonymous union define member functions.

unions with Members of Class Type

Under earlier versions of C++, unions could not have members of a class type that defined its own constructors or copy-control members. Under the new standard, this restriction is lifted. However, unions with members that define their

own constructors and/or copy-control members are more complicated to use than unions that have members of built-in type.

When a union has members of built-in type, we can use ordinary assignment to change the value that the union holds. Not so for unions that have members of nontrivial class types. When we switch the union's value to and from a member of class type, we must construct or destroy that member, respectively: When we switch the union to a member of class type, we must run a constructor for that member's type; when we switch from that member, we must run its destructor.

When a union has members of built-in type, the compiler will synthesize the memberwise versions of the default constructor or copy-control members. The same is not true for unions that have members of a class type that defines its own default constructor or one or more of the copy-control members. If a union member's type defines one of these members, the compiler synthesizes the corresponding member of the union as deleted (§ 13.1.6, p. 508).

For example, the `string` class defines all five copy-control members and the default constructor. If a union contains a `string` and does not define its own default constructor or one of the copy-control members, then the compiler will synthesize that missing member as deleted. If a class has a union member that has a deleted copy-control member, then that corresponding copy-control operation(s) of the class itself will be deleted as well.

Using a Class to Manage union Members

Because of the complexities involved in constructing and destroying members of class type, unions with class-type members ordinarily are embedded inside another class. That way the class can manage the state transitions to and from the member of class type. As an example, we'll add a `string` member to our union. We'll define our union as an anonymous union and make it a member of a class named `Token`. The `Token` class will manage the union's members.

To keep track of what type of value the union holds, we usually define a separate object known as a **discriminant**. A discriminant lets us discriminate among the values that the union can hold. In order to keep the union and its discriminant in sync, we'll make the discriminant a member of `Token` as well. Our class will define a member of an enumeration type (§ 19.3, p. 832) to keep track of the state of its union member.

The only functions our class will define are the default constructor, the copy-control members, and a set of assignment operators that can assign a value of one of our union's types to the union member:

```
class Token {
public:
    // copy control needed because our class has a union with a string member
    // defining the move constructor and move-assignment operator is left as an exercise
    Token(): tok(INT), ival{0} { }
    Token(const Token &t): tok(t.tok) { copyUnion(t); }
    Token &operator=(const Token&);
    // if the union holds a string, we must destroy it; see § 19.1.2 (p. 824)
    ~Token() { if (tok == STR) sval.~string(); }
```

```

    // assignment operators to set the differing members of the union
    Token &operator=(const std::string&);
    Token &operator=(char);
    Token &operator=(int);
    Token &operator=(double);
private:
    enum {INT, CHAR, DBL, STR} tok; // discriminant
    union {                          // anonymous union
        char    cval;
        int     ival;
        double  dval;
        std::string sval;
    }; // each Token object has an unnamed member of this unnamed union type
    // check the discriminant and copy the union member as appropriate
    void copyUnion(const Token&);
};

```

Our class defines a nested, unnamed, unscoped enumeration (§ 19.3, p. 832) that we use as the type for the member named `tok`. We defined `tok` following the close curly and before the semicolon that ends the definition of the `enum`, which defines `tok` to have this unnamed `enum` type (§ 2.6.1, p. 73).

We'll use `tok` as our discriminant. When the union holds an `int` value, `tok` will have the value `INT`; if the union has a `string`, `tok` will be `STR`; and so on.

The default constructor initializes the discriminant and the union member to hold an `int` value of 0.

Because our union has a member with a destructor, we must define our own destructor to (conditionally) destroy the `string` member. Unlike ordinary members of a class type, class members that are part of a union are not automatically destroyed. The destructor has no way to know which type the union holds, so it cannot know which member to destroy.

Our destructor checks whether the object being destroyed holds a `string`. If so, the destructor explicitly calls the `string` destructor (§ 19.1.2, p. 824) to free the memory used by that `string`. The destructor has no work to do if the union holds a member of any of the built-in types.

Managing the Discriminant and Destroying the `string`

The assignment operators will set `tok` and assign the corresponding member of the union. Like the destructor, these members must conditionally destroy the `string` before assigning a new value to the union:

```

Token &Token::operator=(int i)
{
    if (tok == STR) sval.~string(); // if we have a string, free it
    ival = i;                       // assign to the appropriate member
    tok = INT;                      // update the discriminant
    return *this;
}

```

If the current value in the union is a `string`, we must destroy that `string` before assigning a new value to the union. We do so by calling the `string` destructor.

Once we've cleaned up the `string` member, we assign the given value to the member that corresponds to the parameter type of the operator. In this case, our parameter is an `int`, so we assign to `ival`. We update the discriminant and return.

The `double` and `char` assignment operators behave identically to the `int` version and are left as an exercise. The `string` version differs from the others because it must manage the transition to and from the `string` type:

```
Token &Token::operator=(const std::string &s)
{
    if (tok == STR)    // if we already hold a string, just do an assignment
        sval = s;
    else
        new(&sval) string(s);    // otherwise construct a string
    tok = STR;           // update the discriminant
    return *this;
}
```

In this case, if the union already holds a `string`, we can use the normal `string` assignment operator to give a new value to that `string`. Otherwise, there is no existing `string` object on which to invoke the `string` assignment operator. Instead, we must construct a `string` in the memory that holds the union. We do so using placement `new` (§ 19.1.2, p. 824) to construct a `string` at the location in which `sval` resides. We initialize that `string` as a copy of our `string` parameter. We next update the discriminant and return.

Managing Union Members That Require Copy Control

Like the type-specific assignment operators, the copy constructor and assignment operators have to test the discriminant to know how to copy the given value. To do this common work, we'll define a member named `copyUnion`.

When we call `copyUnion` from the copy constructor, the union member will have been default-initialized, meaning that the first member of the union will have been initialized. Because our `string` is not the first member, we know that the union member doesn't hold a `string`. In the assignment operator, it is possible that the union already holds a `string`. We'll handle that case directly in the assignment operator. That way `copyUnion` can assume that if its parameter holds a `string`, `copyUnion` must construct its own `string`:

```
void Token::copyUnion(const Token &t)
{
    switch (t.tok) {
        case Token::INT: ival = t.ival; break;
        case Token::CHAR: cval = t.cval; break;
        case Token::DBL: dval = t.dval; break;
        // to copy a string, construct it using placement new; see (§ 19.1.2 (p. 824))
        case Token::STR: new(&sval) string(t.sval); break;
    }
}
```

This function uses a `switch` statement (§ 5.3.2, p. 178) to test the discriminant. For

the built-in types, we assign the value to the corresponding member; if the member we are copying is a string, we construct it.

The assignment operator must handle three possibilities for its string member: Both the left-hand and right-hand operands might be a string; neither operand might be a string; or one but not both operands might be a string:

```
Token &Token::operator=(const Token &t)
{
    // if this object holds a string and t doesn't, we have to free the old string
    if (tok == STR && t.tok != STR) sval.~string();
    if (tok == STR && t.tok == STR)
        sval = t.sval; // no need to construct a new string
    else
        copyUnion(t); // will construct a string if t.tok is STR
    tok = t.tok;
    return *this;
}
```

If the union in the left-hand operand holds a string, but the union in the right-hand does not, then we have to first free the old string before assigning a new value to the union member. If both unions hold a string, we can use the normal string assignment operator to do the copy. Otherwise, we call `copyUnion` to do the assignment. Inside `copyUnion`, if the right-hand operand is a string, we'll construct a new string in the union member of the left-hand operand. If neither operand is a string, then ordinary assignment will suffice.

EXERCISES SECTION 19.6

Exercise 19.21: Write your own version of the `Token` class.

Exercise 19.22: Add a member of type `Sales_data` to your `Token` class.

Exercise 19.23: Add a move constructor and move assignment to `Token`.

Exercise 19.24: Explain what happens if we assign a `Token` object to itself.

Exercise 19.25: Write assignment operators that take values of each type in the union.

19.7 Local Classes

A class can be defined inside a function body. Such a class is called a **local class**. A local class defines a type that is visible only in the scope in which it is defined. Unlike nested classes, the members of a local class are severely restricted.



All members, including functions, of a local class must be completely defined inside the class body. As a result, local classes are much less useful than nested classes.

In practice, the requirement that members be fully defined within the class limits the complexity of the member functions of a local class. Functions in local classes are rarely more than a few lines of code. Beyond that, the code becomes difficult for the reader to understand.

Similarly, a local class is not permitted to declare `static` data members, there being no way to define them.

Local Classes May Not Use Variables from the Function's Scope

The names from the enclosing scope that a local class can access are limited. A local class can access only type names, `static` variables (§ 6.1.1, p. 205), and enumerators defined within the enclosing local scopes. A local class may not use the ordinary local variables of the function in which the class is defined:

```
int a, val;
void foo(int val)
{
    static int si;
    enum Loc { a = 1024, b };
    // Bar is local to foo
    struct Bar {
        Loc locVal; // ok: uses a local type name
        int barVal;
        void fooBar(Loc l = a) // ok: default argument is Loc::a
        {
            barVal = val; // error: val is local to foo
            barVal = ::val; // ok: uses a global object
            barVal = si; // ok: uses a static local object
            locVal = b; // ok: uses an enumerator
        }
    };
    // ...
}
```

Normal Protection Rules Apply to Local Classes

The enclosing function has no special access privileges to the `private` members of the local class. Of course, the local class could make the enclosing function a friend. More typically, a local class defines its members as `public`. The portion of a program that can access a local class is very limited. A local class is already encapsulated within the scope of the function. Further encapsulation through information hiding is often overkill.

Name Lookup within a Local Class

Name lookup within the body of a local class happens in the same manner as for other classes. Names used in the declarations of the members of the class must be in scope before the use of the name. Names used in the definition of a member can appear anywhere in the class. If a name is not found as a class member, then

the search continues in the enclosing scope and then out to the scope enclosing the function itself.

Nested Local Classes

It is possible to nest a class inside a local class. In this case, the nested class definition can appear outside the local-class body. However, the nested class must be defined in the same scope as that in which the local class is defined.

```
void foo()
{
    class Bar {
    public:
        // ...
        class Nested;      // declares class Nested
    };
    // definition of Nested
    class Bar::Nested {
        // ...
    };
}
```

As usual, when we define a member outside a class, we must indicate the scope of the name. Hence, we defined `Bar::Nested`, which says that `Nested` is a class defined in the scope of `Bar`.

A class nested in a local class is itself a local class, with all the attendant restrictions. All members of the nested class must be defined inside the body of the nested class itself.

19.8 Inherently Nonportable Features

To support low-level programming, C++ defines some features that are inherently **nonportable**. A nonportable feature is one that is machine specific. Programs that use nonportable features often require reprogramming when they are moved from one machine to another. The fact that the sizes of the arithmetic types vary across machines (§ 2.1.1, p. 32) is one such nonportable feature that we have already used.

In this section we'll cover two additional nonportable features that C++ inherits from C: bit-fields and the `volatile` qualifier. We'll also cover linkage directives, which is a nonportable feature that C++ adds to those that it inherits from C.

19.8.1 Bit-fields

A class can define a (nonstatic) data member as a **bit-field**. A bit-field holds a specified number of bits. Bit-fields are normally used when a program needs to pass binary data to another program or to a hardware device.



The memory layout of a bit-field is machine dependent.

A bit-field must have integral or enumeration type (§ 19.3, p. 832). Ordinarily, we use an unsigned type to hold a bit-field, because the behavior of a signed bit-field is implementation defined. We indicate that a member is a bit-field by following the member name with a colon and a constant expression specifying the number of bits:

```
typedef unsigned int Bit;
class File {
    Bit mode: 2;           // mode has 2 bits
    Bit modified: 1;       // modified has 1 bit
    Bit prot_owner: 3;     // prot_owner has 3 bits
    Bit prot_group: 3;     // prot_group has 3 bits
    Bit prot_world: 3;     // prot_world has 3 bits
    // operations and data members of File
public:
    // file modes specified as octal literals; see § 2.1.3 (p. 38)
    enum modes { READ = 01, WRITE = 02, EXECUTE = 03 };
    File &open(modes);
    void close();
    void write();
    bool isRead() const;
    void setWrite();
};
```

The mode bit-field has two bits, modified only one, and the other members each have three bits. Bit-fields defined in consecutive order within the class body are, if possible, packed within adjacent bits of the same integer, thereby providing for storage compaction. For example, in the preceding declaration, the five bit-fields will (probably) be stored in a single unsigned int. Whether and how the bits are packed into the integer is machine dependent.

The address-of operator (&) cannot be applied to a bit-field, so there can be no pointers referring to class bit-fields.



WARNING

Ordinarily it is best to make a bit-field an unsigned type. The behavior of bit-fields stored in a signed type is implementation defined.

Using Bit-fields

A bit-field is accessed in much the same way as the other data members of a class:

```
void File::write()
{
    modified = 1;
    // ...
}
void File::close()
{
    if (modified)
        // ... save contents
}
```

Bit-fields with more than one bit are usually manipulated using the built-in bitwise operators (§ 4.8, p. 152):

```
File &File::open(File::modes m)
{
    mode |= READ;      // set the READ bit by default
    // other processing
    if (m & WRITE) // if opening READ and WRITE
        // processing to open the file in read/write mode
    return *this;
}
```

Classes that define bit-field members also usually define a set of inline member functions to test and set the value of the bit-field:

```
inline bool File::isRead() const { return mode & READ; }
inline void File::setWrite() { mode |= WRITE; }
```

19.8.2 **volatile** Qualifier



The precise meaning of **volatile** is inherently machine dependent and can be understood only by reading the compiler documentation. Programs that use **volatile** usually must be changed when they are moved to new machines or compilers.

Programs that deal directly with hardware often have data elements whose value is controlled by processes outside the direct control of the program itself. For example, a program might contain a variable updated by the system clock. An object should be declared **volatile** when its value might be changed in ways outside the control or detection of the program. The **volatile** keyword is a directive to the compiler that it should not perform optimizations on such objects.

The **volatile** qualifier is used in much the same way as the **const** qualifier. It is an additional modifier to a type:

```
volatile int display_register; // int value that might change
volatile Task *curr_task;     // curr_task points to a volatile object
volatile int iax[max_size];   // each element in iax is volatile
volatile Screen bitmapBuf;    // each member of bitmapBuf is volatile
```

There is no interaction between the **const** and **volatile** type qualifiers. A type can be both **const** and **volatile**, in which case it has the properties of both.

In the same way that a class may define **const** member functions, it can also define member functions as **volatile**. Only **volatile** member functions may be called on **volatile** objects.

§ 2.4.2 (p. 62) described the interactions between the **const** qualifier and pointers. The same interactions exist between the **volatile** qualifier and pointers. We can declare pointers that are **volatile**, pointers to **volatile** objects, and pointers that are **volatile** that point to **volatile** objects:

```

volatile int v;      // v is a volatile int
int *volatile vip;   // vip is a volatile pointer to int
volatile int *ivp;   // ivp is a pointer to volatile int
// vivp is a volatile pointer to volatile int
volatile int *volatile vivp;

int *ip = &v;        // error: must use a pointer to volatile
*ivp = &v;           // ok: ivp is a pointer to volatile
vivp = &v;           // ok: vivp is a volatile pointer to volatile

```

As with `const`, we may assign the address of a volatile object (or copy a pointer to a volatile type) only to a pointer to volatile. We may use a volatile object to initialize a reference only if the reference is volatile.

Synthesized Copy Does Not Apply to volatile Objects

One important difference between the treatment of `const` and `volatile` is that the synthesized copy/move and assignment operators cannot be used to initialize or assign from a volatile object. The synthesized members take parameters that are references to (nonvolatile) `const`, and we cannot bind a nonvolatile reference to a volatile object.

If a class wants to allow volatile objects to be copied, moved, or assigned, it must define its own versions of the copy or move operation. As one example, we might write the parameters as `const volatile` references, in which case we can copy or assign from any kind of `Foo`:

```

class Foo {
public:
    Foo(const volatile Foo&); // copy from a volatile object
    // assign from a volatile object to a nonvolatile object
    Foo& operator=(volatile const Foo&);
    // assign from a volatile object to a volatile object
    Foo& operator=(volatile const Foo&) volatile;
    // remainder of class Foo
};

```

Although we can define copy and assignment for volatile objects, a deeper question is whether it makes any sense to copy a volatile object. The answer to that question depends intimately on the reason for using volatile in any particular program.

19.8.3 Linkage Directives: `extern "C"`

C++ programs sometimes need to call functions written in another programming language. Most often, that other language is C. Like any name, the name of a function written in another language must be declared. As with any function, that declaration must specify the return type and parameter list. The compiler checks calls to functions written in another language in the same way that it handles ordinary C++ functions. However, the compiler typically must generate different

code to call functions written in other languages. C++ uses **linkage directives** to indicate the language used for any non-C++ function.



Mixing C++ with code written in any other language, including C, requires access to a compiler for that language that is compatible with your C++ compiler.

Declaring a Non-C++ Function

A linkage directive can have one of two forms: single or compound. Linkage directives may not appear inside a class or function definition. The same linkage directive must appear on every declaration of a function.

As an example, the following declarations shows how some of the C functions in the `cstring` header might be declared:

```
// illustrative linkage directives that might appear in the C++ header <cstring>
// single-statement linkage directive
extern "C" size_t strlen(const char *);
// compound-statement linkage directive
extern "C" {
    int strcmp(const char*, const char*);
    char *strcat(char*, const char*);
}
```

The first form of a linkage directive consists of the `extern` keyword followed by a string literal, followed by an “ordinary” function declaration.

The string literal indicates the language in which the function is written. A compiler is required to support linkage directives for C. A compiler may provide linkage specifications for other languages, for example, `extern "Ada"`, `extern "FORTRAN"`, and so on.

Linkage Directives and Headers

We can give the same linkage to several functions at once by enclosing their declarations inside curly braces following the linkage directive. These braces serve to group the declarations to which the linkage directive applies. The braces are otherwise ignored, and the names of functions declared within the braces are visible as if the functions were declared outside the braces.

The multiple-declaration form can be applied to an entire header file. For example, the C++ `cstring` header might look like

```
// compound-statement linkage directive
extern "C" {
    #include <string.h>           // C functions that manipulate C-style strings
}
```

When a `#include` directive is enclosed in the braces of a compound-linkage directive, all ordinary function declarations in the header file are assumed to be functions written in the language of the linkage directive. Linkage directives can be

nested, so if a header contains a function with its own linkage directive, the linkage of that function is unaffected.



The functions that C++ inherits from the C library are permitted to be defined as C functions but are not required to be C functions—it's up to each C++ implementation to decide whether to implement the C library functions in C or C++.

Pointers to **extern "C"** Functions

The language in which a function is written is part of its type. Hence, every declaration of a function defined with a linkage directive must use the same linkage directive. Moreover, pointers to functions written in other languages must be declared with the same linkage directive as the function itself:

```
// pf points to a C function that returns void and takes an int
extern "C" void (*pf)(int);
```

When `pf` is used to call a function, the function call is compiled assuming that the call is to a C function.

A pointer to a C function does not have the same type as a pointer to a C++ function. A pointer to a C function cannot be initialized or be assigned to point to a C++ function (and vice versa). As with any other type mismatch, it is an error to try to assign two pointers with different linkage directives:

```
void (*pf1)(int);           // points to a C++ function
extern "C" void (*pf2)(int); // points to a C function
pf1 = pf2; // error: pf1 and pf2 have different types
```



Some C++ compilers may accept the preceding assignment as a language extension, even though, strictly speaking, it is illegal.

Linkage Directives Apply to the Entire Declaration

When we use a linkage directive, it applies to the function and any function pointers used as the return type or as a parameter type:

```
// f1 is a C function; its parameter is a pointer to a C function
extern "C" void f1(void(*) (int));
```

This declaration says that `f1` is a C function that doesn't return a value. It has one parameter, which is a pointer to a function that returns nothing and takes a single `int` parameter. The linkage directive applies to the function pointer as well as to `f1`. When we call `f1`, we must pass it the name of a C function or a pointer to a C function.

Because a linkage directive applies to all the functions in a declaration, we must use a type alias (§ 2.5.1, p. 67) if we wish to pass a pointer to a C function to a C++ function:

```
// FC is a pointer to a C function
extern "C" typedef void FC(int);
// f2 is a C++ function with a parameter that is a pointer to a C function
void f2(FC *);
```

Exporting Our C++ Functions to Other Languages

By using the linkage directive on a function definition, we can make a C++ function available to a program written in another language:

```
// the calc function can be called from C programs
extern "C" double calc(double dparm) { /* ... */ }
```

When the compiler generates code for this function, it will generate code appropriate to the indicated language.

It is worth noting that the parameter and return types in functions that are shared across languages are often constrained. For example, we can almost surely not write a function that passes objects of a (nontrivial) C++ class to a C program. The C program won't know about the constructors, destructors, or other class-specific operations.

PREPROCESSOR SUPPORT FOR LINKING TO C

To allow the same source file to be compiled under either C or C++, the preprocessor defines `__cplusplus` (two underscores) when we compile C++. Using this variable, we can conditionally include code when we are compiling C++:

```
#ifdef __cplusplus
// ok: we're compiling C++
extern "C"
#endif
int strcmp(const char*, const char*);
```

Overloaded Functions and Linkage Directives

The interaction between linkage directives and function overloading depends on the target language. If the language supports overloaded functions, then it is likely that a compiler that implements linkage directives for that language would also support overloading of these functions from C++.

The C language does not support function overloading, so it should not be a surprise that a C linkage directive can be specified for only one function in a set of overloaded functions:

```
// error: two extern "C" functions with the same name
extern "C" void print(const char*);
extern "C" void print(int);
```

If one function among a set of overloaded functions is a C function, the other functions must all be C++ functions:

```
class SmallInt { /* ... */ };
class BigNum { /* ... */ };
// the C function can be called from C and C++ programs
// the C++ functions overload that function and are callable from C++
extern "C" double calc(double);
extern SmallInt calc(const SmallInt&);
extern BigNum calc(const BigNum&);
```

The C version of `calc` can be called from C programs and from C++ programs. The additional functions are C++ functions with class parameters that can be called only from C++ programs. The order of the declarations is not significant.

EXERCISES SECTION 19.8.3

Exercise 19.26: Explain these declarations and indicate whether they are legal:

```
extern "C" int compute(int *, int);
extern "C" double compute(double *, double);
```

CHAPTER SUMMARY

C++ provides several specialized facilities that are tailored to particular kinds of problems.

Some applications need to take control of how memory is allocated. They can do so by defining their own versions—either class specific or global—of the library operator `new` and operator `delete` functions. If the application defines its own versions of these functions, `new` and `delete` expressions will use the application-defined version.

Some programs need to directly interrogate the dynamic type of an object at run time. Run-time type identification (RTTI) provides language-level support for this kind of programming. RTTI applies only to classes that define virtual functions; type information for types that do not define virtual functions is available but reflects the static type.

When we define a pointer to a class member, the pointer type also encapsulates the type of the class containing the member to which the pointer points. A pointer to member may be bound to any member of the class that has the appropriate type. When we dereference a pointer to member, we must supply an object from which to fetch the member.

C++ defines several additional aggregate types:

- Nested classes, which are classes defined in the scope of another class. Such classes are often defined as implementation classes of their enclosing class.
- unions are a special kind of class that may define several data members, but at any point in time, only one member may have a value. unions are most often nested inside another class type.
- Local classes, which are defined inside a function. All members of a local class must be defined in the class body. There are no `static` data members of a local class.

C++ also supports several inherently nonportable features, including bit-fields and `volatile`, which make it easier to interface to hardware, and linkage directives, which make it easier to interface to programs written in other languages.

DEFINED TERMS

anonymous union Unnamed union that is not used to define an object. Members of an anonymous union become members of the surrounding scope. These unions may not have member functions and may not have private or protected members.

bit-field Class member with a integral type that specifies the number of bits to allocate to the member. Bit-fields defined in consecutive order in the class are, if possible, com-

pacted into a common integral value.

discriminant Programming technique that uses an object to determine which actual type is held in a union at any given time.

dynamic_cast Operator that performs a checked cast from a base type to a derived type. When the base type has at least one virtual function, the operator checks the dynamic type of the object to which the refer-

ence or pointer is bound. If the object type is the same as the type of the cast (or a type derived from that type), then the cast is done. Otherwise, a zero pointer is returned for a pointer cast, or an exception is thrown for a cast to a reference type.

enumeration Type that groups a set of named integral constants.

enumerator Member of an enumeration. Enumerators are `const` and may be used where integral constant expressions are required.

free Low-level memory deallocation function defined in `cstdlib`. `free` may be used *only* to free memory allocated by `malloc`.

linkage directive Mechanism used to allow functions written in a different language to be called from a C++ program. All compilers must support calling C and C++ functions. It is compiler dependent whether any other languages are supported.

local class Class defined inside a function. A local class is visible only inside the function in which it is defined. All members of the class must be defined inside the class body. There can be no `static` members of a local class. Local class members may not access the `nonstatic` variables defined in the enclosing function. They may use type names, `static` variables, or enumerators defined in the enclosing function.

malloc Low-level memory allocation function defined in `cstdlib`. Memory allocated by `malloc` must be freed by `free`.

mem_fn Library class template that generates a callable object from a given pointer to member function.

nested class Class defined inside another class. A nested class is defined inside its enclosing scope: Nested-class names must be unique within the class scope in which they are defined but can be reused in scopes outside the enclosing class. Access to the

nested class outside the enclosing class requires use of the scope operator to specify the scope(s) in which the class is nested.

nested type Synonym for nested class.

nonportable Features that are inherently machine specific and may require change when a program is ported to another machine or compiler.

operator delete Library function that frees untyped, unconstructed memory allocated by `operator new`. The library `operator delete[]` frees memory used to hold an array that was allocated by `operator new[]`.

operator new Library function that allocates untyped, unconstructed memory of a given size. The library function `operator new[]` allocates raw memory for arrays. These library functions provide a more primitive allocation mechanism than the library allocator class. Modern C++ programs should use the allocator classes rather than these library functions.

placement new expression Form of `new` that constructs its object in specified memory. It does no allocation; instead, it takes an argument that specifies where the object should be constructed. It is a lower-level analog of the behavior provided by the `construct` member of the allocator class.

pointer to member Pointer that encapsulates the class type as well as the member type to which the pointer points. The definition of a pointer to member must specify the class name as well as the type of the member(s) to which the pointer may point:

```
T C::*pmem = &C::member;
```

This statement defines `pmem` as a pointer that can point to members of the class named `C` that have type `T` and initializes `pmem` to point to the member in `C` named `member`. To use the pointer, we must supply an object or pointer to type `C`:

```
classobj.*pmem;  
classptr->*pmem;
```

fetches member from the object `classobj` of the object pointed to by `classptr`.

run-time type identification Language and library facilities that allow the dynamic type of a reference or pointer to be obtained at run time. The RTTI operators, `typeid` and `dynamic_cast`, provide the dynamic type only for references or pointers to class types with virtual functions. When applied to other types, the type returned is the static type of the reference or pointer.

scoped enumeration New-style enumeration in which the enumerators are not accessible directly in the surrounding scope.

typeid operator Unary operator that returns a reference to an object of the library type named `type_info` that describes the type of the given expression. When the expression is an object of a type that has virtual functions, then the dynamic type of the expression is returned; such expressions are evaluated at run time. If the type is a reference, pointer, or other type that does not define virtual functions, then the type returned is the static type of the reference, pointer, or object; such expressions are not evaluated.

type_info Library type returned by the `typeid` operator. The `type_info` class is inherently machine dependent, but must provide a small set of operations, including a name function that returns a character string representing the type's name. `type_info` objects may not be copied, moved, or assigned.

union Classlike aggregate type that may define multiple data members, only one of which can have a value at any one point. Unions may have member functions, including constructors and destructors. A union may not serve as a base class. Under the new standard, unions can have members that are class types that define their own copy-control members. Such unions obtain deleted copy control if they do not themselves define the corresponding copy-control functions.

unscoped enumeration Enumeration in which the enumerators are accessible in the surrounding scope.

volatile Type qualifier that signifies to the compiler that a variable might be changed outside the direct control of the program. It is a signal to the compiler that it may not perform certain optimizations.